上海交通大学　离散数学课程

实验二：符号执行

实验手册　2026

本实验要求使用 SMT 求解器 Z3 为小型语言 MiniC 实现一个符号执行工具。给定一个 MiniC 程序，该工具要么找出一组触发程序失败的具体输入，要么证明不存在此类输入。课堂上已经介绍了符号执行的基本原理，本实验的目标是将其实现为一个支持完整语言特性、结果可靠且运行高效的分析工具。实验由个人独立完成，允许使用 AI 编程辅助工具，最终提交 C++ 源代码。

1　环境配置

请在 Linux、WSL 或 macOS 环境下完成实验。实验需要支持 C++17 的编译器（如 GCC 12）、GNU make、zip 以及 Python 3。其中 Python 仅用于安装 Z3，实验本身不需要编写或运行 Python 代码。在 Ubuntu / Debian 系统中，可通过以下命令安装基础依赖：

```
sudo apt-get update
sudo apt-get install build-essential python3 python3-venv zip
```

实验使用 `z3-solver` 5.3.1 安装包，其中已包含 C++ 头文件 `z3++.h` 和动态库，无需从源码编译 Z3。建议在 Python 虚拟环境中安装：

```
python3 -m venv .venv
. .venv/bin/activate
pip install -r requirements.txt
```

`Makefile` 会自动定位该安装包中的头文件和库文件。若需使用其他路径下的 Z3，可指定 `Z3_DIR`（例如 `make Z3_DIR=/opt/z3`），此时需确保头文件位于 `$Z3_DIR/include`、库文件位于 `$Z3_DIR/lib`。评测环境仅提供 C++ 标准库与上述版本的 Z3，且无网络连接。

下载并解压实验发布包后，在项目根目录下执行以下命令，检查环境是否正常工作：

```
make
make grade
./build/see tasks/public/p02_warmup_xor.mc
./build/run tasks/public/p02_warmup_xor.mc '{"x": 1, "y": 2}'
```

`make` 会编译生成三个可执行文件：`build/see` 是待实现的符号执行工具，`build/run` 调用参考解释器在指定输入上运行程序，`build/grade` 是本地评测程序。`make grade` 会在 23 道公开测试题上运行评测，并施加与线上评测相同的单题限制（60 秒、2 GiB 内存、1 MiB 输出）。由于起始代码尚未实现核心逻辑，默认全部输出 `unknown`，得分为 0。发布包中提供的 `Dockerfile` 可用于构建与线上评测相同的编译环境（编译器与 Z3 版本）。

Windows 和 macOS 系统的环境配置及 OpenCode 使用方法与实验一相同，请参考实验一手册附录。

2 MiniC 简介

MiniC 的语法接近 C 语言，但语义更加严格，不存在未定义行为。示例如下：

```
int add3(int x) {
    return x + 3;
}

void check(int x, int key[8]) {
    assume(key[0] == 'K');
    assert(add3(x) != 10);
}
```

程序的入口函数为 `check`，其参数即为程序的输入。MiniC 的主要特性如下（完整规范见附录 A，建议在开始编写代码前通读）：

- 仅支持两种基本类型：`int`（32 位有符号整数）和 `bool`，以及它们的定长数组。两种类型之间不能互相转换。
- 整数运算采用 32 位补码回绕语义，溢出不会报错。`/` 和 `%` 的行为与 C99 一致：商向零截断，余数的符号与被除数相同。
- 唯一的循环结构为 `for i in LO..HI`（其中 LO、HI 为常量），且不支持递归调用。因此，所有 MiniC 程序都必然终止。
- 没有全局变量和指针，数组参数按值传递。
- `assert(c)` 表示待验证的断言性质；`assume(c)` 表示输入需满足的前提条件。

给定一组具体输入，程序执行共有四种结局：

结局	是否算失败	触发条件
`ok`	否	`check` 正常返回
`assert_fail`	**是**	某处 `assert(c)` 的条件 `c` 为假
`trap`	**是**	除数为 0、-2^{31} 除以 -1、数组下标越界，或移位量不在 $0 \sim 31$ 之间
`blocked`	否	某处 `assume(c)` 的条件 `c` 为假，当前执行路径被忽略

语言语义以参考解释器为准。发布包中的 `minic/interp.cpp` 给出了 MiniC 语义的完整参考实现，评测系统同样使用该解释器判定一组输入是否触发程序失败。若对语言细节存有疑问，可直接通过 `build/run` 运行程序加以验证。

3 符号执行原理

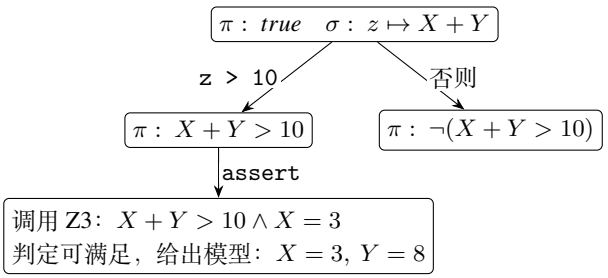
3.1 基本思路

直接运行程序时，输入是具体的数，各变量在运行过程中取的也都是具体的数值。运行一次只能知道一组输入是否会导致程序失败；而输入空间通常非常庞大（例如两个 32 位 `int` 参数就有 2^{64} 种组合），逐一尝试是不现实的。

符号执行（Symbolic Execution）则换了一种思路：它不传入具体数值，而是把每个输入参数都看作一个**符号变量**（如 X、Y），带着这些符号去推演程序的执行过程。在推演到程序某处时，当前的**符号状态**由两部分组成：

- **符号存储** σ：记录每个变量当前的符号值。这不再是一个具体的数，而是关于输入符号的表达式（如 $X+Y$）。
- **路径条件** π：记录程序要想执行到当前位置，输入符号所必须满足的约束条件。

```
void check(int x, int y) {
    int z = x + y;
    if (z > 10) {
        assert(x != 3);
    }
}
```



以上面的程序为例，符号执行的过程如下：

1. 执行 `int z = x + y;` 时，不再计算具体数值，而是在符号存储中记录 $\sigma(z) = X + Y$。
2. 遇到分支 `if (z > 10)` 时，条件 $X + Y > 10$ 的真假取决于输入。执行在这里**分叉**（fork）成两个状态：一个路径条件为 $\pi \wedge (X + Y > 10)$，进入 `if` 分支；另一个为 $\pi \wedge \neg(X + Y > 10)$，跳过 `if` 分支。
3. 在 `if` 分支内遇到 `assert(x != 3)` 时，该断言被违反当且仅当输入既能走到这里、又使 $x = 3$ 成立，即公式 $X + Y > 10 \wedge X = 3$ 可满足。调用 SMT 求解器 Z3 判定该公式，Z3 回答可满足并给出**模型**（满足赋值）$X = 3, Y = 8$——这就找到了一个触发断言失败的具体输入。

可以看出，直接运行程序每次只跟踪一个具体的输入，而符号执行中的一个状态刻画了所有满足路径条件 π 的输入。遇到条件分支时，状态分叉相当于把这些输入按条件一分为二，分别推演。

程序能否失败。程序中每处可能触发运行时错误的操作（如 `assert`、除法、数组下标访问）都有对应的**失败条件** $fail$。设 π 为到达该处的路径条件，则

$$\text{程序能失败} \iff \exists \text{ 某处操作，使得 } \pi \wedge fail \text{ 可满足。}$$

符号执行的任务就是遍历所有状态，在每处可能出错的操作上调用 Z3 检查此公式。

前提条件。`assume(c)` 不产生失败条件，而是将路径条件更新为 $\pi \wedge c$：不满足前提的输入将被过滤，其后续的失败与当前推演无关。在 `assume` 之前发生的错误仍然算作失败，因为那时条件 c 尚未加入 π。

3.2 算法框架

将上一节的思路转化为具体算法，即得到图 1 所示的整体结构。白框代表符号执行器自身的处理流程，分为三个阶段；灰框代表外部的 Z3 求解器，负责判定逻辑公式的可满足性。

第 1 步：建立符号。为入口函数 `check` 的每个参数创建对应的 Z3 符号变量；数组参数的每个元素分别创建一个独立的符号变量（命名形如 `key[0]`）。初始状态的符号存储 σ 将各参数映射至对应的符号变量，初始路径条件设为 $\pi = \mathit{true}$。

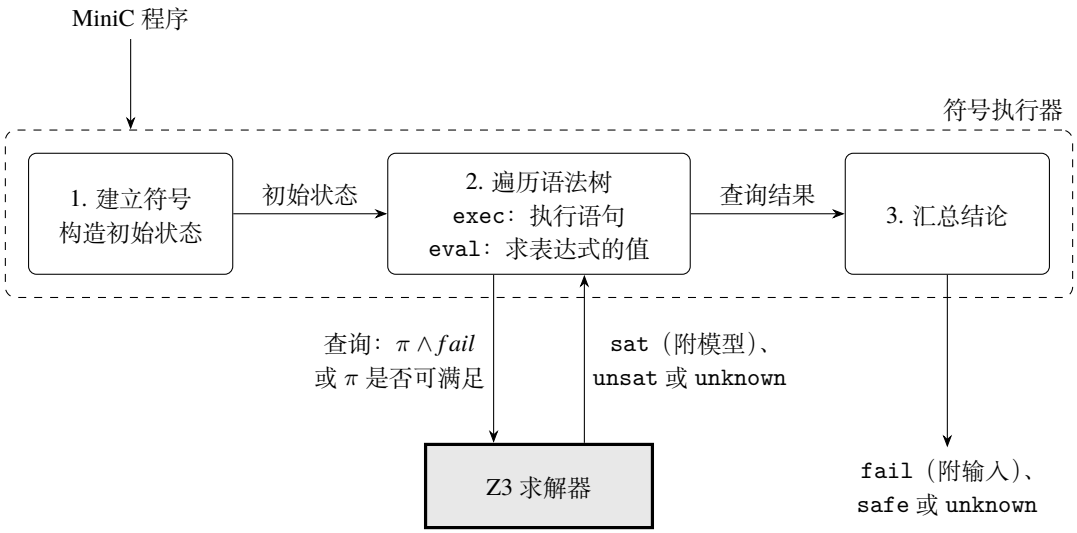


图 1: 符号执行器的整体结构。白框为待实现的算法模块，灰框为外部求解器。

第 2 步：遍历语法树。算法的核心是由两个相互递归调用的函数组成的语法树遍历过程：

- `exec(s, st)`：在状态 `st` 上执行语句 `s`，返回执行后产生的**所有**后续状态。由于 `if` 语句会导致分支分叉，返回的状态可能有多个；若当前路径在断言等位置必然失败或被过滤，返回的状态集合也可能为空。
- `eval(e, st)`：在状态 `st` 上对表达式 `e` 求值，所得的符号值是一个 Z3 表达式（例如当 $\sigma(x) = X$ 时，`x + 1` 的求值结果为 $X + 1$）。由于表达式内部可能包含引发分叉的函数调用，`eval` 会返回一组结果，每个结果包含一个更新后的状态及该状态下的表达式值（详见第 3.6 节）；当表达式不含函数调用时，仅返回一个结果。

`exec` 在执行语句时会调用 `eval` 计算其中的子表达式。设 `eval` 求得的符号值为 v，各类基本语句的执行规则如下：

语句	**`exec` 的处理方式**
`int x = e;`、`x = e;`	更新符号存储 $\sigma(x) = v$
`assert(c)`	检查失败条件 $\neg v$
`assume(c)`	更新路径条件 $\pi = \pi \wedge v$
`{ s1 s2 ... }`	依次执行块内语句；前一条语句产生的每个状态，均作为下一条语句的输入状态继续执行
`if`、`for`	见第 3.5 节

遍历过程中共有两类针对 Z3 的可满足性查询：

- **失败检查** `check(st, fail)`：到达可能触发错误的位置时，调用 Z3 判定 $\pi \wedge fail$ 是否可满足。若可满足，则表明找到了触发失败的具体路径，保存求解器给出的模型并终止整个分析过程；若不可满足，则说明当前位置在该路径下不会出错，继续后续执行，并将路径条件收紧为 $\pi \wedge \neg fail$（排除已确认安全的反面情况）。
- **可行性剪枝** `feasible(st)`：状态分叉后，调用 Z3 判定新状态的 π 是否可满足。若不可满足，说明不存在任何能够到达该分支的输入，可直接丢弃该状态。剪枝仅用于提升执行效率，不会改变最终结论——当 π 不可满足时，后续任何 $\pi \wedge fail$ 也必然不可满足。

Z3 查询还可能返回 `unknown`（例如求解超时）。此时不能将其当作不可满足处理：在剪枝时应保留该状态继续探索；在失败检查时需记录存在未决查询，最终不能输出 `safe`。

第 3 步：汇总结论。语法树遍历结束后，根据收集到的信息得出最终结论：

- 若已记录满足失败条件的模型：输出 `fail`，并从模型中提取各输入符号的具体取值构造反例输入。注意 Z3 中 32 位位向量的取值为 $0 \sim 2^{32}-1$ 的无符号位模式，需按补码转换回有符号整数。
- 若未找到反例模型，但遍历过程中曾有查询返回 `unknown`，或遇到了未实现的语言特性：输出 `unknown`。
- 其余情况：输出 `safe`。此时所有潜在失败位置均已检查完毕，且对应的失败条件均被证明不可满足。

接下来的几节将逐步引入 MiniC 的各项语言特性。处理每项特性时，都需要明确三个核心问题：该特性的值在 Z3 中如何表示；`exec` 或 `eval` 遇到它时如何更新状态；它是否可能触发运行时错误，对应的失败条件是什么。

3.3 位向量

前两节把 $X+Y$ 当作数学上的加法，而 Z3 的 `Int` 类型也对应数学整数，不会发生溢出。但 MiniC 的 `int` 是 32 位的，`2147483647 + 1` 应当回绕为 -2^{31}。因此，必须使用 Z3 的**位向量**（bit-vector）类型来准确建模：使用 `ctx.bv_const(name, 32)` 创建一个 32 位位向量常量，`ctx.bool_const(name)` 创建布尔常量。下文中的 `ctx` 指代 `z3::context` 对象。

位向量的编码方案与课程所学的 eager SMT 思路一致：Z3 将每个 32 位变量展开为 32 个命题变项，将算术运算编码为对应的逻辑电路，最后交给 SAT 求解器处理。使用位向量建模的另一个优势在于，它将取值空间限制在有限范围内，避免了使用数学整数编码时处理非线性运算（如两个变量相乘）可能导致的求解失败。

运算的翻译。`eval` 遇到运算时，先求出操作数的符号值，再调用相应的 Z3 运算。位向量本身只是一串比特，不分有符号和无符号，区别在于比较、除法、右移等运算逻辑。MiniC 的 `int` 类型运算对应关系如下：

MiniC	Z3 C++	说明
`+ - * & | ^`、一元 `- ~`	`a + b` 等	按 2^{32} 回绕，与 MiniC 一致
`< <= > >=`	`a < b` 等	C++ 运算符在位向量上执行有符号比较
`== !=`、`!`	`a == b` 等	同样适用于 `bool`
`/`	`a / b`	有符号除法，向零截断
`%`	`srem(a, b)`	**不能写** `a % b`，详见下文
`<<`、`>>`	`shl(a, s)`、`ashr(a, s)`	**不能写** `a << s`、`a >> s`，详见下文

注意以下两点细节：

- **取余**。Z3 C++ 接口中位向量的 `a % b` 对应 `bvsmod`，余数符号跟随**除数**；而 MiniC 要求余数跟随**被除数**（$-7 \bmod 2 = -1$），故需使用 `srem`。
- **移位**。`z3::expr` 提供了隐式转换为 `bool` 的 `operator bool()`，但未重载 `<<` 和 `>>`。若直接写 `a << s`，C++ 会先将两个表达式隐式转换为布尔值再做整数移位，既能通过编译，又不会产生预期的位向量移位表达式。因此必须调用 `shl`、`ashr`。

运行时错误。Z3 的运算符在逻辑上总能得出结果（例如除数为 0 时返回 -1）。但在 MiniC 中，除数为 0、除法溢出（$-2^{31}/-1$）、移位量越界等情况均为 **trap**。因此，每个此类运算都是一个失败位置，需显式定义失败条件：

运算	失败条件
`a / b`、`a % b`	$b = 0 \vee (a = -2^{31} \wedge b = -1)$
`a << s`、`a >> s`	$s < 0 \vee s > 31$

正确的处理流程是：先求出操作数的值，检查失败条件，最后才计算运算结果。检查通过后，除数不为 0 且移位量合法，此时 Z3 的运算语义与 MiniC 保持一致。

3.4 数组

数组长度是常量，因此可用 `std::vector<z3::expr>` 表示一个数组，其中每个元素均为一个 Z3 表达式。在局部数组声明时，所有元素初始化为 0（整数）或 `false`（布尔）。

读写操作。若下标为常量，直接访问对应位置的元素即可。在循环展开后，由于循环变量在每次迭代中均为常量，绝大多数下标访问均为常量。若下标为符号值 i，则由于其取值不确定，需将所有潜在的读写位置编码进表达式：

- **读取** `a[i]`：结果为 $\text{ite}(i = 0, a_0, \text{ite}(i = 1, a_1, \ldots, a_{N-1}))$，使用嵌套的 `z3::ite` 逻辑。
- **写入** `a[i] = v`：每个元素 a_j 在写入后，新值更新为 $\text{ite}(i = j, v, a_j)$。

另一种可选的编码方式是使用 Z3 的数组理论：利用 `z3::select` 进行读取，`z3::store` 进行写入。两种编码方式在不同场景下各有性能优势，可在后续性能优化阶段进行比较。

越界检查。每次数组读写操作均是潜在的失败位置。对于长度为 N 的数组，失败条件为 $i < 0 \vee i \geq N$。需注意下标可能为负，必须使用有符号比较逻辑。在实际读写操作前，应先检查该条件。

状态复制。状态在进行分支分叉时会按值复制，随之数组结构也会完整复制。因此，不同分支中的写入操作互不干扰。`z3::expr` 是引用计数的句柄，故此种数组结构的复制开销较小。

3.5 分支与循环

`if` 语句。首先求出条件表达式的值 v，随后将当前状态分叉为两份：一份在路径条件 $\pi \wedge v$ 下执行 then 分支；另一份在 $\pi \wedge \neg v$ 下执行 else 分支（若无 else 则直接结束）。两个分支执行前需先进行剪枝检查。所有分支输出的状态集合，即为 `if` 语句的执行结果。若条件表达式中包含函数调用，`eval` 可能返回多个结果，此时需对每一个结果分别执行上述过程。

短路求值。对于 `a && b`，仅当 `a` 为真时才计算 `b`，因此 `b` 中的潜在失败操作仅在 $\pi \wedge a$ 成立时才可能触发。例如 `assert(y == 0 || 10 / y < 100)` 中，除法操作仅在 $y \neq 0$ 时执行。必须在 $\pi \wedge y \neq 0$ 的约束下检查除法失败，此时结果不可满足。若忽略短路语义，可能导致误报错误。实现时，在计算右操作数之前，需将左操作数的结果并入路径条件。

循环。`for i in LO..HI` 的执行次数已知，故可完全展开：令循环变量 i 依次取值 $\text{LO}, \ldots, \text{HI} - 1$，每次将其作为常量代入循环体中执行。

`break`、`continue` 与 `return`。这些语句会改变程序的正常控制流。为此，状态中设有一个 `flow` 字段，用于记录当前的控制流去向：执行到 `break` 时，`flow` 变为 `Break`，当前语句块中的后续语句将跳过不执行。该状态沿语句块向上层传递，直至被外层的循环或函数调用捕获并处理：

- `Continue`：视作当前轮次的循环体执行完毕，将 `flow` 重置为 `Normal` 后进入下一轮迭代。
- `Break`：跳出当前循环，将 `flow` 重置为 `Normal` 后继续执行循环之后的语句。
- `Return`：直接跳出当前循环及所在函数，交由函数调用逻辑提取返回值。

若某轮迭代后不再有进入下一轮的状态（例如所有状态均已 `break` 或 `return`），即可提前结束循环展开。

注意，每个 `if` 都可能使状态数翻倍。循环展开后，循环体内的 `if` 会变成多次连续的分支，状态数可能随迭代次数呈指数增长。第 3.7.1 节将介绍状态合并技术来解决这一问题。

3.6 函数调用

由于 MiniC 不支持递归，函数调用可直接通过**内联**（inlining）方式处理：遇到 `f(e1, e2)` 时，首先在调用者状态下依次求出各实参的值，随后构造一个新的符号存储（仅包含形参到实参值的映射），并沿用调用者当时的路径条件去执行被调函数的函数体。

由于没有全局变量和指针，且数组按值传递，被调函数只能修改其自身的局部变量，不会改变调用者的符号存储。但路径条件会随着函数体的执行而更新（例如函数体内包含 `if` 分支或 `assume` 语句）。当执行到 `return e` 时，状态的 `flow` 被设为 `Return` 并记录返回值。函数体执行完毕后，每个到达末尾的状态都代表函数调用的一种可能结果：将调用者原有的符号存储与该状态演化后的路径条件结合，并取出记录的返回值，即可继续外层表达式的求值。

这也解释了为何 `eval` 在处理函数调用时会返回一组 `Result`：

```
int abs(int v) {
    if (v < 0) { return -v; }
    return v;
}
// 调用处：assert(abs(x) + 1 > 0);
```

在上述例子中，`eval` 求值 `abs(x)` 会产生两个结果：一个路径条件为 $\pi \wedge X < 0$、返回值为 $-X$；另一个路径条件为 $\pi \wedge \neg(X < 0)$、返回值为 X。外层的加法运算和断言检查将在每个结果上分别继续执行。

3.7 性能优化

上面的算法已经能正确处理所有 MiniC 程序，但未必能在每题 60 秒的限时内完成。最主要的问题是状态数的指数增长。

3.7.1 状态合并

```
void check(int x) {
    int n = 0;
    for i in 0..32 {
        if (((x >> i) & 1) == 1) { n = n + 1; }
    }
    assert(n <= 32);
}
```

循环展开后有 32 个连续的 `if`，导致状态数高达 2^{32}，无法逐一检查。但这些状态到达断言时，仅有变量 `n` 的值不同。解决方法是在控制流的**汇合点**（join point）将多个状态合并为一个，用条件表达式记录变量在不同分支中的取值。

合并规则。设 $(\pi_1, \sigma_1), \ldots, (\pi_k, \sigma_k)$ 为到达同一位置且路径条件两两互斥的状态，合并后的状态为

$$\pi = \pi_1 \vee \cdots \vee \pi_k, \qquad \sigma(x) = \mathrm{ite}\big(\pi_1, \sigma_1(x), \mathrm{ite}(\pi_2, \sigma_2(x), \ldots, \sigma_k(x))\big).$$

合并后的状态刻画了满足 π 的所有输入，各输入的变量取值由 ite 表达式根据所属分支选出。**合并时使用的是各分支结束时的路径条件**。对于 `if (c)`，设分叉前的路径条件为 π，c 为分叉时求得的条件。仅当两个分支正常到达汇合点时的路径条件仍分别等价于 $\pi \wedge c$ 和 $\pi \wedge \neg c$ 时，合并后的路径条件才能简化回 π，变量取值为 $\mathrm{ite}(c, \sigma_{\text{then}}(x), \sigma_{\text{else}}(x))$。图 2 展示了这一过程。

由于分支内的 `assume` 或失败检查可能进一步过滤输入，不能仅凭两个分支均正常执行完毕就直接恢复分叉前的路径条件。例如，初始路径条件为 *true* 时，`if (x > 0) { assume(x > 10); }` 的两个分支结束条件分别为 $X > 10$ 和 $X \le 0$，合并后应为 $X > 10 \vee X \le 0$，而非 *true*。

合并的代价。合并后状态数不再增长，但变量的取值变为更复杂的条件表达式，相当于将原本由状态分叉完成的搜索转移给了 Z3。前文的计数程序在合并后始终只有一个状态，`n` 成为一个 32 层的 `ite` 表达式，Z3 可以很快证明断言成立。

```
int s = 0;
if (x > 0) {
    s = x;
} else {
    s = -x;
}
assert(s >= 0);
```

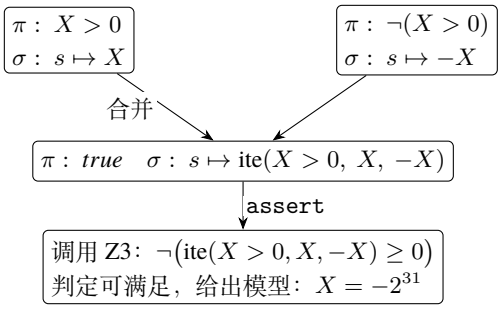


图 2: `if` 的两个分支在汇合处合并，断言只需检查一次。$-(-2^{31})$ 在 32 位下回绕为 -2^{31}，因此断言失败；若使用数学整数编码，则会误判为安全。

合并范围。仅合并处于同一位置且 `flow` 相同的状态：`Normal`、`Break`、`Continue`、`Return` 各自合并，其中 `Return` 状态的返回值 `ret` 同样使用 ite 合并。语句块结束时，块内声明的局部变量已离开作用域，因此到达同一汇合点的状态具有相同的变量集合。

合并后可省去剪枝。如第 3.2 节所述，剪枝仅影响速度而不改变结论。引入状态合并后，状态数不再呈指数级增长，分支处的 `feasible` 查询往往得不偿失：即使将不可满足的状态一并合并，也只是在 π 中增加一个恒假的析取项。省去这些查询通常能显著提升运行速度。

3.7.2 其他优化

- **增量求解**。利用 `push` 和 `pop` 复用已加入求解器的路径条件，避免每次查询都重新构造求解器。
- **常量折叠**。当两个操作数均为常量时，直接在 C++ 中计算结果，无需构造复合 Z3 表达式。整数运算可先用 `int64_t` 计算，再调用 `minic::wrap` 回绕至 `int` 范围；除法需先检查 trap。
- **表达式化简**。调用 `e.simplify()` 可以精简表达式，但化简本身也有开销，需通过实测权衡。
- **编码与查询策略**。例如，数组采用 `ite` 还是数组理论；失败条件是逐处查询，还是汇总为合取析取公式 $\bigvee_p(\pi_p \wedge fail_p)$ 后一次性求解。
- **查询超时**。为单次求解设置超时限制（见附录 B）。超时的查询返回 `unknown`，不能视作不可满足。

优化时建议每次只调整一个因素，在相同题集上对比耗时，并验证结论的正确性。

4 实验内容

4.1 任务

工具的运行命令为 `./build/see TASK.mc`。给定 MiniC 程序 `TASK.mc`，工具需判断该程序是否存在导致失败的输入，并在**标准输出的最后一行**打印一个 JSON 对象。三种回答格式如下：

```
{"verdict": "fail", "input": {"x": -5, "key": [75, 0, 0, 0, 0, 0, 0, 0]}}
{"verdict": "safe"}
{"verdict": "unknown"}
```

其中 `input` 字段须完整包含 `check` 函数的所有参数。整数需为 JSON 格式的 32 位有符号整数，布尔值为 `true` 或 `false`，数组则表示为长度匹配的列表。调试信息请输出至标准错误流；标准输出总量不得超过 1 MiB，超出按格式错误计 0 分。起始代码已实现上述输出格式。请注意第 5.1 节的计分规则：若工具误将程序判定为 `safe`，将扣除 2 分；对于无法确定的情况，请输出 `unknown`。

4.2 起始代码

发布包的主要内容如下：

路径	说明
see/core.cpp	**待实现的核心代码**，详见第 4.3 节
see/engine.h、see/engine.cpp	符号执行框架（含状态定义、语法树遍历等）。无须修改，建议通读
see/main.cpp	主程序：负责读题、调用框架、验证反例并输出答案
minic/	语言前端及参考解释器（包含词法、语法、类型检查、解释器、输入校验等）。接口在 minic.h，语法树定义在 ast.h
tasks/public/	23 道公开测试题，expected.json 为参考答案
tools/	build/run 和 build/grade 的源码，评测规则与线上保持一致

see/ 目录支持添加新的源文件，Makefile 会自动编译该目录下的所有 .cpp 文件。注意，线上评测环境使用的是独立的 minic/ 副本，因此**提交中修改的 minic/ 代码会被忽略**。

代码结构。图 3 展示了工具的代码结构。实线框为已实现的框架代码，虚线框为需补充的函数，底色标示了该函数所属的功能模块（见第 4.3 节）。

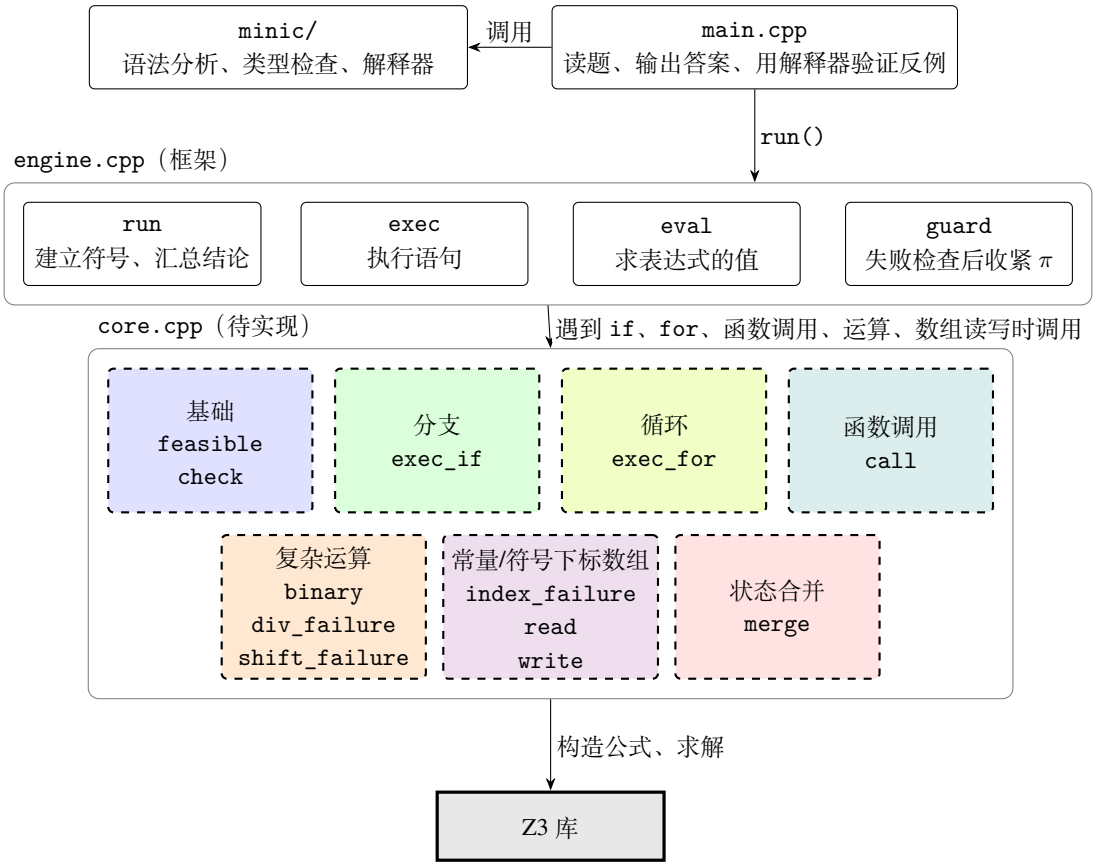


图 3: 符号执行工具的代码结构图。

框架代码的逻辑对应第 3.2 节描述的三步：

- run：负责建立符号（第 1 步）及遍历结束后的结论汇总（第 3 步）。找到失败时，由 counterexample 将 Z3 模型转为输入。
- exec 和 eval：处理与语法树结构相关的逻辑（第 2 步），如语句顺序执行、变量作用域、赋值、断言、假设、控制流，以及表达式求值与操作前的失败检查。
- guard：执行失败检查（调用 check），并将路径条件 π 更新为 $\pi \wedge \neg \mathit{fail}$；若失败条件恒真，直接丢弃该状态。

- `main.cpp`：在输出 `fail` 前，使用解释器对反例进行验证。若验证未失败，则修正结果为 `unknown` 并输出 `BUG` 提示，这通常表明符号编码与语义不一致。

待实现的函数均位于 `core.cpp` 中。在实现之前，这些函数默认抛出 `Unsupported` 异常，使工具输出 `unknown`（不扣分）。编写代码时也应保持这一约定：遇到无法处理的构造时抛出 `Unsupported`；当 Z3 返回 `unknown` 时将 `incomplete_` 设为 `true`。这两种情况最终都会使工具输出 `unknown`，避免误判为 `safe`。

状态表示。`engine.h` 中的 `State` 包含四个字段：路径条件 `pc`（Z3 布尔表达式）、符号存储 `store`、控制流状态 `flow`（取值为 `Normal`、`Break`、`Continue` 或 `Return`，见第 3.5 节），以及 `Return` 状态携带的返回值 `ret`。`store` 将变量名映射至 `Value`（即 `std::vector<z3::expr>`）：标量变量仅含一个元素，可通过 `store.at("x")[0]` 读取；长度为 N 的数组则包含 N 个元素。

框架接口与成员变量。在 `core.cpp` 中会用到以下接口与成员：

- `exec(stmt, st)`：在状态 `st` 上执行单条语句（或语句块），返回执行后的所有状态。该函数仅对 `flow` 为 `Normal` 的状态执行语句，其余状态原样返回。
- `eval(expr, st)`：对表达式求值，返回一组 `Result`（每个元素包含 `state` 和 `value` 两个字段）。
- `ctx_`：Z3 上下文对象，可用于构造常量，如 `ctx_.bv_val(5, 32)`、`ctx_.bool_val(true)`。
- `model_`、`incomplete_`：找到触发失败的路径时，将模型存入 `model_`；若有查询返回未知结果，将 `incomplete_` 设为 `true`。

抽象语法树。语法树定义在 `minic/ast.h` 中。语句节点为 `minic::Stmt`，通过 `kind` 字段区分类型，各节点使用的具体字段已在头文件注释中标明。例如：`for` 语句的循环变量名为 `s.name`，上下界为 `s.lo` 和 `s.hi`，循环体为 `s.body`；`if` 语句的条件为 `s.cond`，两个分支分别为 `s.then_block` 和 `s.else_block`（无 `else` 分支时为空指针）。函数节点为 `minic::Func`，包含参数列表 `params` 和函数体 `body`。

4.3 功能模块与计分

`core.cpp` 中的待实现函数分为 8 个功能模块。每个模块对应一组题目，这组题目只用到该模块及其前置模块的特性，按模块独立计分：模块得分 = 模块分值 × 该模块题目的得分比例（单题计分见第 5.1 节）。总分为各模块得分之和，上限 100 分。基础模块 40 分，其余模块各 20 分，因此**完成基础模块和任意三个模块即可获得满分**。图 4 给出模块之间的依赖关系：箭头指向前置模块，做一个模块之前要先做完它的所有前置模块。

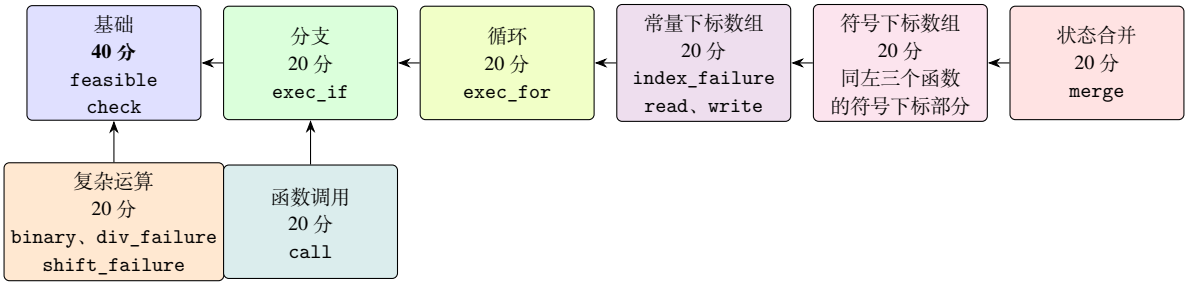


图 4: 功能模块的依赖关系与分值。箭头指向前置模块。

公开题也按模块分组，运行 `make grade` 可看到每个模块的得分，`make grade BLOCK=loop` 则只评测某一模块的题目：

模块	待实现函数	公开题	分值
基础	`feasible`、`check`	p01–p03	40
分支	`exec_if`	p04–p06	20
复杂运算	`binary`（除、取余、移位）、`div_failure`、`shift_failure`	p07–p10	20
函数调用	`call`	p11、p12	20
循环	`exec_for`	p13–p15	20
常量下标数组	`index_failure`、`read`、`write` 的常量下标部分	p16–p18	20
符号下标数组	同上三个函数的符号下标部分	p19、p20	20
状态合并	`merge`	p21–p23	20

基础（第 3.2 节）。`feasible` 判定状态的路径条件是否可满足，`check` 判定某处的失败条件是否可能成立，两者均需按第 3.2 节的要求处理 `unknown`。完成后即可处理只有顺序语句、`assume` 和 `assert` 的程序。

分支（第 3.5 节）。`exec_if` 在条件分支处分叉状态，并丢弃不可行的分叉。

复杂运算（第 3.3 节）。补全 `binary` 中的除法、取余和移位运算，并实现除法与移位的失败条件。这一模块容易在边界细节上出错，常见现象是触发 `main.cpp` 的 `BUG` 提示。

函数调用（第 3.6 节）。`call` 在新状态中内联执行被调函数体，并把每种返回方式的结果带回调用处。

循环（第 3.5 节）。`exec_for` 展开循环，并根据 `flow` 处理 `break`、`continue` 和 `return`。

常量下标数组（第 3.4 节）。当下标是常量（例如循环变量）时，`index_failure` 直接给出越界与否，`read` 和 `write` 直接访问对应元素。

符号下标数组（第 3.4 节）。当下标依赖输入时，用嵌套 `ite` 表达式或 Z3 数组理论编码越界条件和读写操作。

状态合并（第 3.7.1 节）。在 `exec_if`、`exec_for` 和 `call` 的控制流汇合点合并状态。该模块的题目是状态机、排序等不合并就会超时的程序。合并之后变量的值可能变成 ite 表达式，原本为常量的数组下标也随之变为符号下标，所以这一模块以符号下标数组为前置。

每个模块的题目只在该模块及其前置模块都做对时才能解出；未实现的模块只会让对应题目输出 `unknown`，不会影响其他模块的得分。排行榜上的用时差距主要取决于第 3.7 节的优化。

4.4 调试与测试

差分测试。发布包自带的参考解释器是最有效的调试工具。可以手动编写或随机生成小型 MiniC 程序，通过以下方式进行交叉验证：

- 当工具输出 `fail` 时，调用解释器验证生成的反例输入是否确实触发失败（`main.cpp` 已内置这一检查）；
- 在 `check` 开头使用 `assume` 将各 `int` 参数限制在较小范围内（如 $-8 \sim 7$），从而枚举所有可能的输入，利用解释器得出精确真值，并与工具输出的 `safe` 或 `fail` 进行对比。

单元测试。每实现一个新语法特性，建议编写若干仅包含该特性的小程序进行测试，并重点覆盖边界情形：-2^{31}、$2^{31}-1$、移位量 31、32 与 -1、负数下标，以及恰好等于数组长度的下标。

公开题评测。编译后运行 `make grade` 可在全部公开题上进行评测，`make grade BLOCK=loop` 只评测某一模块的题目。直接运行 `./build/grade` 可使用更多选项：添加 `-v` 参数可查看失败用例的标准错误输出，也可以只给出若干题目文件；`./build/grade --help` 列出全部选项。公开题仅供开发调试与基本检查使用，线上隐藏题包含不同的程序，请勿针对公开题的代码文本做特殊处理。

5　评测与提交

5.1　计分规则

每道题独立计分，规则如下：

输出结果	判定条件	得分
`fail`	给出的输入经参考解释器运行，确实触发失败	$+1$
`fail`	输入格式无效，或经解释器运行未触发失败	0
`safe`	程序在所有合法输入下确实均不会失败	$+1$
`safe`	程序实际上存在触发失败的输入（误报安全）	$\mathbf{-2}$
`unknown`、超时、崩溃、输出格式错误	—	0

题目按第 4.3 节的功能模块分组。模块得分 = 模块分值 × 该模块所有题目得分之和与题目数之比（可以为负），总分为各模块得分之和，下限 0 分、上限 100 分。

隐藏测试集中包含大量成对设计的题目：同一份程序有两个相近版本，一份存在漏洞，另一份完全安全，两者仅相差一个边界条件，且触发漏洞的输入极难通过随机测试命中（例如仅当校验和恰好匹配，或输入恰好为某个特定十位数字串时才会触发错误）。因此，“找不到反例就直接输出 `safe`”的策略在隐藏测试集上会导致严重扣分。凡是未能完整证明安全的情况，请务必输出 `unknown`。

5.2　线上评测

实验网站使用隐藏测试集进行评测。隐藏题的考察范围与公开题一致，但具体程序、参数规模及题目数量均不相同。每道题目的运行时间上限为 60 秒（单核），内存上限为 2 GiB，标准输出上限为 1 MiB。评测系统反馈各模块得分与总分，不展示单题结果与具体输出。

提交次数不限，每次提交都会在全部隐藏题上实时评测。排行榜首先按总分高低排序，总分相同时按总用时由短到长排序。截止时以每位同学最后一次提交的得分作为实验的评测成绩。具体的成绩权重、截止时间、评测网址及个人访问令牌（token）请以课程通知为准。

5.3　提交要求

只需修改并提交 `see/` 目录下的源代码。评测平台将使用发布包中的原始 `Makefile` 与平台自带的 `minic/` 副本重新编译你的代码。工具仅允许依赖 C++ 标准库与 Z3 库，严禁调用其他求解器或外部程序。工具只能读取命令行指定的 `TASK.mc`。严禁以任何方式读取、篡改或绕过评测环境（包括读取其他题目文件或答案、干预评分程序、访问其他同学的提交等）；严禁通过匹配题目名称、文件哈希或查表等方式直接返回结果。若参考或使用了外部代码，须在源码注释中注明出处。

完成开发与本地测试后，在项目根目录下依次执行：

```
make
make grade
make handin
```

`make handin` 会将 `see/` 目录下的全部源文件打包生成 `handin.zip`。上传前请检查确认压缩包内未包含个人访问令牌或 API 密钥等敏感信息。随后登录实验评测网站，输入个人令牌并上传 `handin.zip`，即可查看编译与评测进度。

A MiniC 语言规范

本附录给出 MiniC 语言的完整规范。MiniC 的语法虽然接近 C 语言，但**其精确语义完全由本附录及 `minic/interp.cpp` 定义**，请勿凭 C 语言的习惯直接推断。若发现文档描述与参考解释器存在出入，请以解释器行为为准，并向助教反馈。

A.1 程序结构

一个 MiniC 程序由若干函数定义组成（示例见第 2 节）。程序的入口函数名必须为 `check`，返回类型为 `void`，其参数列表即为程序的外部输入。

- 不支持直接或间接递归调用，且 `check` 函数不可被其他函数调用。
- 不设全局变量、指针及动态堆内存分配。
- 支持单行注释 `//` 与块注释 `/* */`。

A.2 类型系统

类型	取值范围	说明
`int`	$-2^{31} \ldots 2^{31}-1$	32 位有符号整数（二进制补码表示）
`bool`	`true`、`false`	布尔类型，与整数类型严格区分

定长数组声明格式为 `T name[N]`，其中元素类型 `T` 为 `int` 或 `bool`，数组长度 `N` 为 $1 \sim 4096$ 之间的十进制整数字面量。

`int` 与 `bool` 之间不能隐式或显式互相转换。二元运算符的左右操作数类型必须一致，否则无法通过静态类型检查。若需将 `bool` 转为整数，可使用 `if` 语句；若需将整数转为 `bool`，可写作比较表达式 `x != 0`。

A.3 字面量

- 整数字面量包括十进制（如 `123`）、十六进制（如 `0x7F`）和字符字面量（如 `'A'`），类型均为 `int`。字符字面量取其 ASCII 码值，支持转义序列 `\n`、`\t`、`\r`、`\0`、`\\`、`\'` 及十六进制转义 `\xNN`。十进制非零字面量不允许包含前导零。
- 所有字面量的值均须落在 `int` 的表示范围内，越界属于静态语法错误。十六进制字面量同样遵循此规则：例如 `0x7FFFFFFF` 合法，而 `0x80000000` 非法。
- 紧跟在一元负号 `-` 后的整数字面量在语法分析阶段会直接合并为负数常量，因此 `-2147483648` 是合法的字面量。

A.4 语句

```
int x = e;              // 标量声明，必须显式初始化
int a[8];               // 数组声明，元素自动全部初始化为 0 或 false
x = e;
a[i] = e;
if (c) { ... } else if (d) { ... } else { ... }   // 分支体必须带有花括号
for i in 0..10 { ... }                            // 循环变量依次取 0, 1, ..., 9
break; continue; return; return e;
assert(c);
assume(c);
f(x, a);                // 仅函数调用可单独作为表达式语句
{ ... }                 // 复合语句块
```

- `for i in LO..HI`：上下界 LO、HI 均为十进制整数字面量，满足 $0 \le \text{LO} \le \text{HI} \le 1000000$，循环共执行 $\text{HI} - \text{LO}$ 次。循环变量 `i` 的类型为 `int`，作用域仅限循环体内，且在循环体内为只读变量。**这是 MiniC 唯一的循环构造**，从而保证了所有程序必然终止。
- 变量采用词法块作用域，禁止变量遮蔽（shadowing）：内层作用域不得声明与外层当前可见变量同名的变量，但互不包含的并列代码块之间可以使用相同变量名。
- 函数形参可视作普通局部变量，允许重新赋值。数组参数采用**按值传递**，函数调用时会复制整个数组，被调函数内对数组元素的修改不会影响调用者原有的数组。
- 返回类型非 `void` 的函数，其所有控制流路径均须以 `return e;` 结束（由前端静态检查保证）。

A.5 表达式与运算符

运算符优先级与 C 语言相同，下表按优先级从高到低排列：

运算符	操作数类型	结果类型与说明
`-e`、`~e`	`int`	`int`
`!e`	`bool`	`bool`
`*`、`/`、`%`	`int`	`int`
`+`、`-`	`int`	`int`
`<<`、`>>`	`int`	`int`
`<`、`<=`、`>`、`>=`	`int`	`bool`
`==`、`!=`	相同类型（`int` 或 `bool`）	`bool`
`&`	`int`	`int`
`^`	`int`	`int`
`|`	`int`	`int`
`&&`	`bool`	`bool`（短路求值）
`||`	`bool`	`bool`（短路求值）

注意，与 C 语言一致，表达式 `x & 1 == 0` 会被解析为 `x & (1 == 0)`，这在 MiniC 中会因类型不匹配而报错。

A.6 运算精确语义

- `+ - *`、一元 `-`、`~`、`& | ^`：均按 32 位二进制补码回绕（即模 2^{32}），无未定义行为。例如：`2147483647 + 1 == -2147483648`，`65536 * 65536 == 0`，`-(-2147483648) == -2147483648`。
- **关系比较**：均为 32 位有符号整数比较。
- **`/` 与 `%`**：商**向零截断**，余数符号与**被除数**保持一致（同 C99 标准），例如 `-7 / 2 == -3`，`-7 % 2 == -1`，`7 % -2 == 1`。当除数为 0 时触发 **trap**；当计算 `-2147483648 / -1` 或 `-2147483648 % -1`（结果溢出）时同样触发 **trap**。
- **`<<` 与 `>>`**：移位量必须位于 $0 \sim 31$ 之间，否则触发 **trap**。左移 `<<` 在低位补 0，移出的高位直接丢弃（例如 `1 << 31 == -2147483648`）；右移 `>>` 为算术右移，高位补符号位（例如 `-8 >> 1 == -4`，`-1 >> 31 == -1`）。
- **数组下标访问 `a[i]`**：当下标不满足 $0 \le i < N$ 时触发 **trap**。

A.7 求值顺序

- 二元运算：先求左操作数，再求右操作数，最后检查 trap 并执行运算。逻辑与 `&&`、逻辑或 `||` 遵循短路求值：若左操作数已能确定结果，则右操作数不予求值，其中潜在的 trap 也不会触发。

- 函数调用：按从左到右的顺序依次求各实参的值，随后进入被调函数。
- 数组元素赋值 `a[i] = e;`：先求下标 `i`，再求右值 `e`，**最后**检查下标 `i` 是否越界并完成写入。
- 数组元素读取 `a[i]`：先求下标 `i`，再检查下标是否越界并读取元素。

严格的求值顺序决定了多种特殊情况同时存在时的最终结局。例如在 `a[100] = e;` 中，若计算 `e` 时调用的函数执行了 `assume(false)`，由于 `e` 的求值早于下标越界检查，该次执行的最终结局为 `blocked` 而非越界 `trap`（四种结局的定义见第 2 节）。

A.8　输入 JSON 格式

程序输入表示为单个 JSON 对象，其键名对应 `check` 函数的形参名，且必须**恰好**包含全部形参：

```
{"x": -5, "flag": true, "key": [65, 66, 45, 49, 50, 51, 52, 53]}
```

- `int` 参数必须表示为 JSON 整数（不得写作 `1.0`、`1e3` 或字符串），且数值须在 32 位有符号整数范围内。
- `bool` 参数必须表示为 `true` 或 `false`，不得使用 0、1 代替；反之，整数参数也不得使用布尔字面量。
- 数组参数必须表示为长度恰好等于声明长度 `N` 的 JSON 数组。
- JSON 对象中不允许出现重复键名，也不允许使用 `NaN` 或 `Infinity`。

凡不符合上述格式要求的输入均视为无效输入。

A.9　程序规模限制

所有评测用例均满足以下规模限制（前端类型检查器会自动拒绝超出限制的程序），符号执行工具可直接依赖这些前提：

- 函数总数不超过 64 个，函数调用链深度不超过 16 层；
- 数组声明长度在 $1 \sim 4096$ 之间，循环上界不超过 1000000；`check` 的全部参数以 JSON 写出（整数按最长 11 位计）不超过 1000000 字节，因此任何反例都能装进 1 MiB 的输出上限；
- 按最坏情况静态估算，单次具体执行的总步数不超过 5000000 步；
- 源文件大小不超过 1 MB，括号、一元运算及代码块的嵌套深度不超过 60 层，抽象语法树总深度不超过 150。

B　Z3 C++ 接口速查

下表列出的常用接口均来自头文件 `#include <z3++.h>`，位于命名空间 `z3` 下（表中 `ctx` 表示 `z3::context` 实例）。完整接口说明请参阅 Z3 C++ API 官方文档。

接口	功能说明
`ctx.bv_const("x", 32)`、`ctx.bool_const("b")`	创建 32 位位向量符号常量、布尔符号常量（即逻辑公式中的符号变量）
`ctx.bv_val(v, 32)`、`ctx.bool_val(true)`	创建具体常量值（`bv_val` 会自动按 2^{32} 取模）
`a + b`、`a < b`、`a == 3` 等	算术运算与有符号比较；其中一侧操作数可直接传入 C++ 整型常量

接口（续）	功能说明
`ite(c, a, b)`	构造条件表达式 $\mathrm{ite}(c, a, b)$
`a && b`、`a || b`、`!a`、`implies(a, b)`	布尔逻辑连接词
`srem(a, b)`	有符号取余（余数符号跟随被除数）；注意不能写 `a % b`
`shl(a, s)`、`ashr(a, s)`	逻辑左移、算术右移；注意不能写 `<<`、`>>`
`e.simplify()`	对表达式进行代数与逻辑化简
`e.is_true()`、`e.is_false()`、`e.is_numeral()`	判断表达式是否为字面常量 `true`、`false` 或具体数值
`z3::eq(a, b)`	判断两个表达式在语法结构上是否完全一致（常数时间，不调用求解器）
`select(arr, i)`、`store(arr, i, v)`	数组理论读写操作；可通过 `ctx.array_sort(...)` 创建数组类型

接口（续）	功能说明
`z3::solver s(ctx);`	创建求解器实例
`s.add(f)`、`s.check()`	向求解器添加约束公式；执行求解，返回 `z3::sat`、`z3::unsat` 或 `z3::unknown`
`s.get_model()`	当求解结果为 `sat` 时，获取满足赋值模型
`s.push()`、`s.pop()`	保存当前约束栈状态、回退至最近一次保存的状态
`z3::params p(ctx); p.set("timeout", 5000u); s.set(p);`	设置单次 `check()` 调用的超时时间（单位：毫秒）
`m.eval(e, true)`	在模型 `m` 下求表达式 `e` 的具体值（第二个参数 `true` 表示对无关变量自动补全赋值）
`v.get_numeral_uint64()`	提取位向量常量的底层位模式（返回无符号 64 位整数）
`v.is_true()`	判断布尔常量表达式是否为真

下面是一个完整的独立示例，用于求解满足 `-x == x` 且 `x != 0` 的 32 位有符号整数 `x`：

```
#include <iostream>
#include <z3++.h>

int main() {
    z3::context ctx;
    z3::expr x = ctx.bv_const("x", 32);
    z3::solver s(ctx);
    s.add(-x == x && x != 0);
    if (s.check() == z3::sat) {
        int64_t v = s.get_model().eval(x, true).get_numeral_uint64();
        if (v >= (int64_t(1) << 31)) v -= int64_t(1) << 32;   // int: signed
        std::cout << v << "\n";                               // -2147483648
    }
}
```